arm64 qemu crash工具使用流程

2024年8月24日 10:10

Author:茆寒

date：2022.8.24

编译内核：

如果crash工具里gdb版本不高，7.3.0工具的gdb版本低，crash时候会报错误，需要指定编译：

```
make KCFLAGS="-gdwarf-4" -j16
```

启动qemu：

```
qemu-system-aarch64 \
-M virt \
-cpu cortex-a57 \
-smp 1 \
-m 512M \
-kernel /home/maohan/linux/kernel/qemu-k54/qemu-k54/arch/arm64/boot/Image \
-nographic \
--append "noinitrd root=/dev/vda rw nokaslr console=ttyAMA0 loglevel=8" \
-drive if=none,file=rootfs_ext4.img,id=hd0 \
-device virtio-blk-device,drive=hd0 \
-netdev user,id=eth0 \
-device virtio-net-device,netdev=eth0 \
-fsdev local,security_model=passthrough,id=fsdev0,path=/home/maohan/linux/kernel/qemu-learn/image \
-device virtio-9p-device,id=fs0,fsdev=fsdev0,mount_tag=hostshare \
```

注：上面两行-fsdev是共享目录，用来传文件

参数解释：

kaslr是内核启动随机地址，vmlinux的内核地址是固定的，所以gdb调试时候要关掉，方法：

1、是重新编译内核，在arch/arm64/configs/defconfig 中将CONFIG_RANDOMIZE_BASE=y修改成CONFIG_RANDOMIZE_BASE=n

2、qemu启动的cmdline中增加nokaslr 参数，通过参数方式关闭

启动虚拟机之后：

echo 1 > /proc/sys/kernel/panic_on_oops 设置panic不重启

echo c > /proc/sysrq-trigger 触发dump

按ctrl + a c打开QEMU控制台，使用以下命令导出vmcore：

(qemu) dump-guest-memory /your_path/vmcore

(qemu) dump-guest-memory -z /your_path/vmcore # 压缩，导出vmcore

安装crash工具：

网站： git://github.com/crash-utility/crash.git

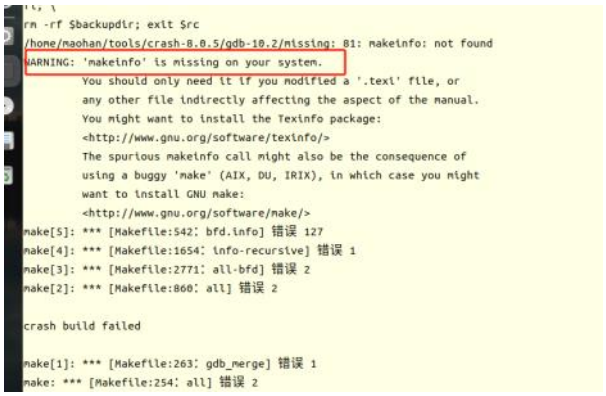
我下载的：为了兼容kernel5.4

安装：解压之后进入文件夹

make target=ARM64

strip -s crash ===》清除掉多余符号，把crash文件复制到自己的coredump文件夹

如果安装8.0.5版本可能make时候会出现：



需要更新：sudo apt-get install libaio-dev libncurses5-dev zlib1g-dev liblzma-dev flex bison byacc

需要安装：sudo apt-get install texinfo

启动crash工具进行分析：

把vmlinux和vmcore和crash工具放到一起：

```
./crash vmcore vmlinux -m vabits_actual=48 -m kimage_voffset=0xffff7fffd0000000
./crash803 vmcore vmlinux -m vabits_actual=48 -m kimage_voffset=0xffff7fffd0000000
```

使用gcc-linaro-13.0.0-2022.11-x86_64_aarch64-linux-gnu编译的kernel 启动crash会出现错误：crash neither runqueue nor rq structures exist，换成gcc-arm-8.3-2019.03-x86_64-aarch64-linux-gnu编译的就ok了

vabits_actual通过查看：

虚拟地址长度可以在.config中查看（64位平台通常的配置是48或者39）：

//CONFIG_ARM64_VA_BITS_48=y CONFIG_ARM64_VA_BITS=48

```
CONFIG_ARM64_4K_PAGES=y
# CONFIG_ARM64_16K_PAGES is not set
# CONFIG_ARM64_64K_PAGES is not set
# CONFIG_ARM64_VA_BITS_39 is not set
CONFIG_ARM64_VA_BITS_48=y
CONFIG_ARM64_VA_BITS=48
CONFIG_ARM64_PA_BITS_48=y
CONFIG_ARM64_PA_BITS=48
```


kimage_voffset可以通过GDB单步调试获取： https://zhuanlan.zhihu.com/p/667555796

首先在qemu启动时候加上-s -S

```
qemu-system-aarch64 \
    -machine virt,virtualization=true,gic-version=3 \
    -nographic \
    -m size=1024M \
    -cpu cortex-a72 \
    -smp 2 \
    -kernel Image \
    -drive format=raw,file=rootfs.img \
    -append "root=/dev/vda rw" \
    -s \
    -S
```

```
# -s 是-gdb tcp::1234 的简写，如果需要换端口可以用-gdb tcp::1234替换-s参数
# -S 是freeze cpu at startup的指令，也就是kernel 启动时就挂起，等待调试连接，如果不需要调试内
    可以去掉
```

GDB单步调试找到镜像虚拟地址： https://blog.csdn.net/tanxjian/article/details/121889366

先安装arm64能用的gdb（貌似arm32的也能用）：

```
apt-get install gdb-multiarch
```

注：p /x kimage_voffset会出现：no debugging symbols found的报错，是因为.config里CONFIG_DEBUG_INFO没开，编译kernel时候会有[Y/N]的提示，全打开来

启动qemu之后再另开一个终端：

先进入编译出vmlinux的kernel文件夹

```
gdb-multiarch vmlinux
(gdb) target remote :1234 ===》连接上qemu的端口
Remote debugging using :1234
0x0000000040000000 in ?? ()
(gdb) b start_kernel ===》设置断点
(gdb) c ===》运行到断点
(gdb) p /x kimage_voffset ===》查看镜像地址
$3 = 0xffff80003fe00000
```

获取完成之后crash：

```
NOTE: setting kimage_voffset to: 0xffff7fffd0000000

GNU gdb (GDB) 7.6
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "--host=x86_64-unknown-linux-gnu --target=aarch64-elf-linux"...

      KERNEL: vmlinux
    DUMPFILE: vmcore  [PARTIAL DUMP]
        CPUS: 1
        DATE: Sat Aug 24 01:12:29 CST 2024
      UPTIME: 00:01:54
LOAD AVERAGE: 0.00, 0.00, 0.00
       TASKS: 46
    NODENAME: (none)
     RELEASE: 5.4.0-g93164cac5-dirty
     VERSION: #6 SMP PREEMPT Fri Aug 23 15:59:11 CST 2024
     MACHINE: aarch64  (unknown Mhz)
      MEMORY: 512 MB
       PANIC: ""
         PID: 0
     COMMAND: "swapper/0"
        TASK: ffff800011792d40  [THREAD_INFO: ffff800011792d40]
         CPU: 0
       STATE: TASK_RUNNING (ACTIVE)
     WARNING: panic task not found

crash> bt
```

如果是通过加载ko模块的进行测试的，在crash命令行下输入：

```
mod -s kdump /kdump/demo/stack/kdump.ko
mod -s <模块的名字，lsmod就知道了>   <ko的文件>
```